

Utilisation de l'objet PDO en PHP

1. Objectif

PDO (PHP Data Objects) est une **interface orientée objet** pour accéder à différentes bases de données avec une **API unique**, sécurisée, et moderne.

PDO permet de se connecter à **MySQL**, **PostgreSQL**, **SQLite**, **SQL Server**, etc., **sans changer beaucoup de code**.

2. Connexion à la base de données

✓ Exemple MySQL :

```
try {
    $pdo = new PDO("mysql:host=localhost;dbname=ma_base;charset=utf8",
    "root", "password");
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION); //
    Activer les exceptions
} catch (PDOException $e) {
    echo "Erreur de connexion : " . $e->getMessage();
}
```

3. Requêtes simples (sans variables)

Exemple : lire des données

```
$sql = "SELECT * FROM utilisateurs";
$result = $pdo->query($sql);

foreach ($result as $row) {
    echo $row['nom'] . "<br>";
}
```

`query()` est utilisé pour les requêtes **sans variables utilisateur** (sinon → injection SQL possible !).

4. Requêtes préparées (avec variables)

✓ Avec `prepare()` et `execute()`

```
$sql = "SELECT * FROM utilisateurs WHERE email = :email";
$stmt = $pdo->prepare($sql);
$stmt->execute(['email' => 'test@example.com']);
$user = $stmt->fetch(PDO::FETCH_ASSOC);
```

Sécurise les données en les échappant automatiquement → contre l'**injection SQL**.

5. Insert, Update, Delete

+ Insertion

```
$sql = "INSERT INTO utilisateurs (nom, email) VALUES (:nom, :email)";
$stmt = $pdo->prepare($sql);
$stmt->execute([
    'nom' => 'Alice',
    'email' => 'alice@example.com'
]);
```

Mise à jour

```
$sql = "UPDATE utilisateurs SET nom = :nom WHERE id = :id";
$stmt = $pdo->prepare($sql);
$stmt->execute(['nom' => 'Alice Dupont', 'id' => 3]);
```

✗ Suppression

```
$sql = "DELETE FROM utilisateurs WHERE id = :id";
$stmt = $pdo->prepare($sql);
$stmt->execute(['id' => 3]);
```

6. Boucle de résultats

```
$stmt = $pdo->query("SELECT * FROM utilisateurs");
while ($row = $stmt->fetch(PDO::FETCH_ASSOC)) {
    echo $row['nom'] . "<br>";
}
```

7. Paramètres de connexion utiles

```
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION); //
Exceptions pour erreurs
$pdo->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC); //
Résultats en tableaux associatifs
```

8. Transactions

```
try {
    $pdo->beginTransaction();

    $pdo->exec("INSERT INTO comptes VALUES (NULL, 'Bob', 100)");
    $pdo->exec("INSERT INTO comptes VALUES (NULL, 'Alice', 150)");

    $pdo->commit();
} catch (PDOException $e) {
    $pdo->rollBack();
    echo "Erreur : " . $e->getMessage();
}
```

9. Bonnes pratiques

Bonne pratique	Pourquoi ?
<code>prepare()</code> + <code>execute()</code>	Sécurise les requêtes (évite injections SQL)
<code>try/catch</code> + <code>PDOException</code>	Gère les erreurs proprement
<code>FETCH_ASSOC</code> par défaut	Résultats lisibles sans doublons
Utiliser <code>bindParam()</code> si besoin de référence	Précision sur les types, utile pour boucles
Utiliser des transactions pour opérations critiques	Cohérence des données garantie

Mauvaises pratiques

Mauvaise pratique	Risque
Concaténer des variables dans la requête (" <code>WHERE id = \$id</code> ")	Vulnérable à l'injection SQL
Ne pas gérer les erreurs (<code>try/catch</code>)	Debug difficile, crash silencieux
Oublier de fermer les curseurs (<code>\$stmt->closeCursor()</code>)	Problèmes avec requêtes multiples (surtout MySQL)
Mélanger affichage HTML + logique PDO	Ruine la séparation MVC, code difficile à maintenir

Mauvaise pratique**Risque**

Ne pas normaliser le charset (ex: UTF-8)

Problèmes d'encodage

10. 📦 Exemple complet (MVC-friendly)

```
// model/UserModel.php
class UserModel {
    private $pdo;

    public function __construct(PDO $pdo) {
        $this->pdo = $pdo;
    }

    public function getAll() {
        $stmt = $this->pdo->query("SELECT * FROM utilisateurs");
        return $stmt->fetchAll();
    }
}

// index.php
require 'model/UserModel.php';

$pdo = new PDO("mysql:host=localhost;dbname=test;charset=utf8", "root",
    "");
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$model = new UserModel($pdo);
$users = $model->getAll();

foreach ($users as $u) {
    echo $u['nom'] . "<br>";
}
```

✅ Résumé

Fonction	Utilité
PDO	Accès universel à plusieurs BDD
prepare() / execute()	Sécurité + performance
fetch() / fetchAll()	Accès aux résultats
beginTransaction() / commit() / rollBack()	Opérations atomiques
setAttribute()	Personnaliser le comportement PDO